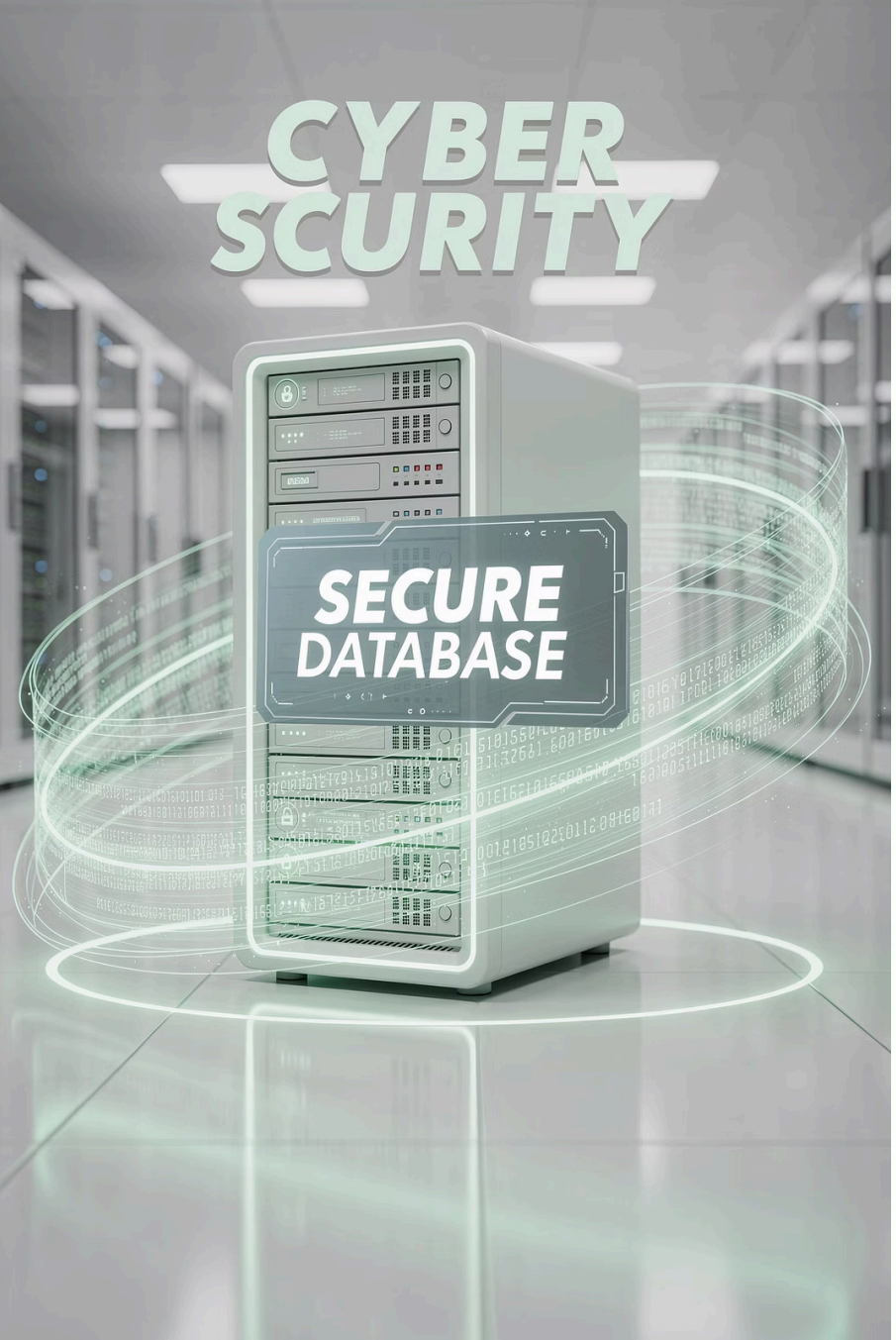


SQL Injection: Unmasking the Hidden Threat

A deep dive into one of the most pervasive and dangerous web vulnerabilities – how attackers exploit the gap between user input and database commands, and what you can do to stop them.

SECURITY SQL WEB VULNERABILITIES

CHAPTER 1

The Invisible Doorway

What is SQL Injection?

Every web application that stores and retrieves data has a conversation with a database – and SQL injection is what happens when an attacker eavesdrops on that conversation and starts speaking for themselves. This chapter lays the groundwork: what SQL injection is, why it exists, and how understanding the basics is the first step toward building safer applications.

The Foundation: How Websites Talk to Databases

SQL as the Common Language

SQL – Structured Query Language – is the universal language that web applications use to communicate with relational databases. Whether you're logging in, searching for a product, or submitting a form, SQL queries are being executed behind the scenes to fetch, store, or update data in real time.

Almost every modern website relies on a database. E-commerce platforms query product catalogs. Banking apps retrieve transaction records. Social networks store and serve user profiles. SQL is the backbone of all of it.

The Role of User Input

The critical link between the user and the database is **user input**. When you type a search term or enter login credentials, that input is typically embedded directly into a SQL query that gets sent to the database server.

A simple login query might look like this:

```
SELECT * FROM users
WHERE username = 'alice'
AND password = 'secret123';
```

The application constructs this query dynamically, inserting whatever the user typed. This dynamic construction is powerful – and, when done carelessly, extremely dangerous. If the application doesn't validate or sanitize that input, an attacker can manipulate the query itself.

The Vulnerability: When Input Becomes Code

SQL Injection (SQLi) is a **code injection attack** in which an attacker inserts or "injects" malicious SQL code into a data entry field. Instead of the application treating the input as plain data, the database interprets it as executable SQL commands – fundamentally altering the query's intent.

How It Happens

When a developer writes code like `query = "SELECT * FROM users WHERE name = '" + userInput + "'"`, they are directly concatenating user-supplied text into a SQL statement. If `userInput` contains special characters like a single quote (`'`) or comment sequences (`--`), the database may interpret them as SQL syntax, breaking out of the intended data context and executing attacker-controlled commands.

The Core Problem: Trust

The root cause of SQL injection is **implicit trust in user input**. The application assumes the data it receives is benign. Attackers exploit this assumption by crafting inputs that contain SQL metacharacters, turning the data channel into a command channel. This class of vulnerability has appeared on the OWASP Top 10 list for decades and continues to cause major data breaches worldwide.

Potential Consequences

A successful SQL injection attack can lead to **full database compromise** – including unauthorized data access, data manipulation or deletion, authentication bypass, and in some cases, remote code execution on the underlying server. The consequences range from a minor data leak to a catastrophic breach affecting millions of users.



Exploiting the Cracks

SQL Injection doesn't break down the door – it slips through the crack that was always there, hidden in plain sight inside every unprotected input field. Understanding where these cracks form is the first step to sealing them.

CHAPTER 2

Simple Injections

A First Look

Before attackers attempt complex multi-step exploits, they often start with simple, elegant injections. These entry-level attacks reveal just how little it takes to compromise an unprotected application – sometimes a single character is all it takes. This chapter walks through two foundational injection scenarios every developer and security professional should understand.

Example 1: Bypassing Login Forms

One of the most classic and illustrative SQL injection scenarios is the login bypass. This attack requires no knowledge of any valid username or password – it exploits the logical structure of the SQL query itself.

Vulnerable Query

```
SELECT * FROM users
WHERE username = 'userInput'
AND password = 'passwordInput';
```

The application directly embeds the user's input into the query string without any sanitization or parameterization.

Attacker's Input (Username)

```
' OR '1'='1
```

The attacker enters this as the username. The single quote closes the string literal, and the OR clause introduces a condition that is always true.

Resulting Query

```
SELECT * FROM users
WHERE username = ''
OR '1'='1'
AND password = 'passwordInput';
```

The injected OR '1'='1' evaluates to TRUE for every row in the database, causing the query to return all users.

Outcome: The application receives a valid result set and treats the first returned user (often an admin) as authenticated – granting full access with no valid credentials whatsoever. This is why input sanitization is non-negotiable in authentication logic.

Example 2: Revealing Data with a Simple Trick

Beyond bypassing authentication, SQL injection can be used to extract data the attacker was never meant to see. This example demonstrates how a `UNION SELECT` injection can append attacker-controlled data to a legitimate query's output – and serve as a gateway to far more destructive extraction techniques.

Vulnerable Query

```
SELECT product_name, price
FROM products
WHERE id = userInput;
```

This query is typical of a product page: the URL parameter `id` is passed directly into the SQL statement. There is no validation of what `userInput` contains.

Attacker's Input

```
1 UNION SELECT 'Hacked', '999'
```

The attacker appends a `UNION SELECT` clause that returns two arbitrary string values, matching the two columns (`product_name` and `price`) in the original query.

Resulting Query

```
SELECT product_name, price
FROM products
WHERE id = 1
UNION SELECT 'Hacked', '999';
```

What This Demonstrates

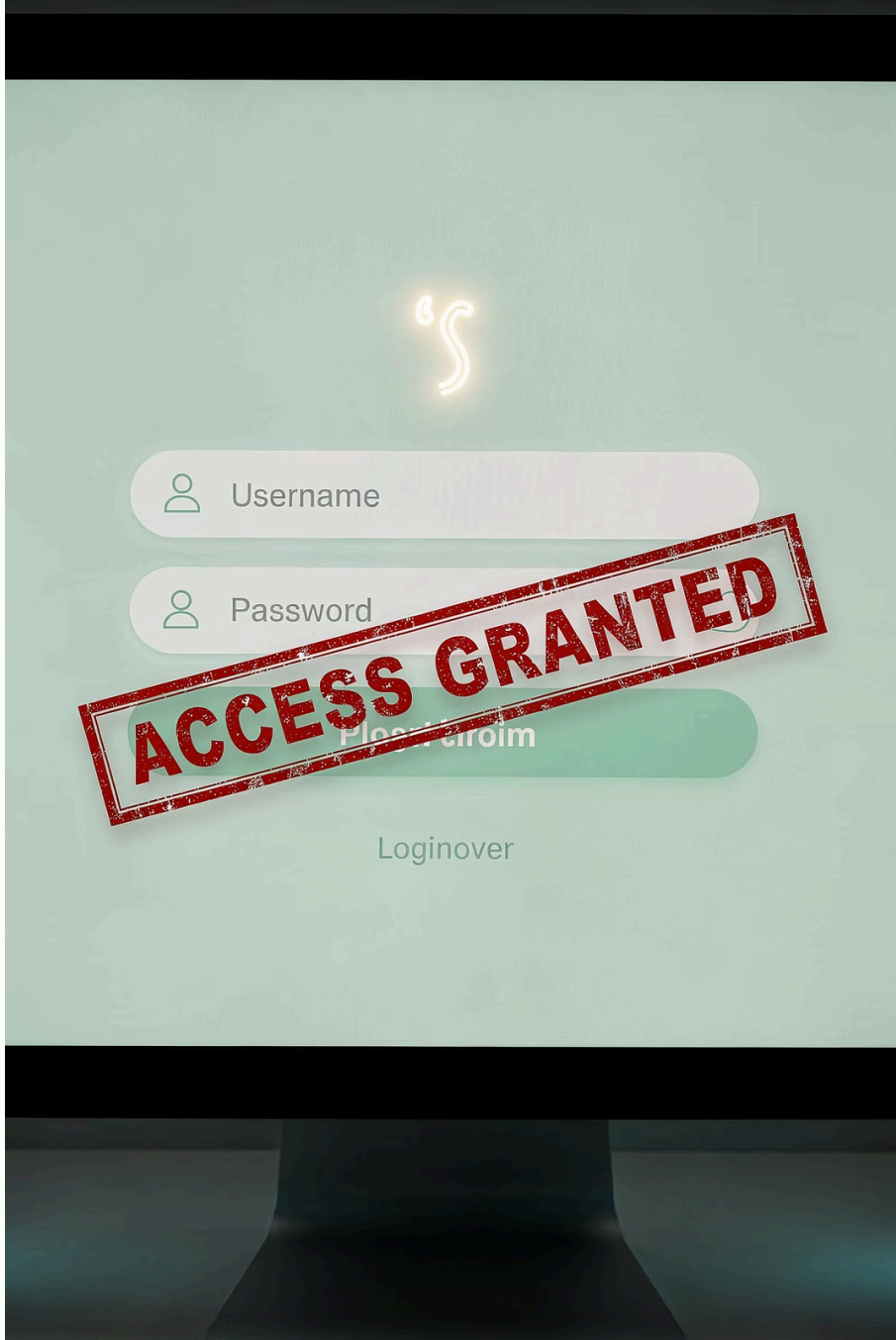
The response from the server will now include a row showing `Hacked | 999` alongside real product data. While this particular example is harmless, it **proves two critical things**:

- The application is injectable – the attacker can append arbitrary SQL
- The attacker knows the number of columns in the original query
- The `UNION` technique works and can be escalated to pull real sensitive data

From here, replacing `'Hacked', '999'` with real column references from real tables – like `username, password FROM users` – becomes the next step in a full data extraction attack.

The Power of a Single Quote

A single apostrophe (') is often all it takes. This tiny character, when inserted into an unguarded input field, can shatter the boundary between data and code – handing an attacker the keys to your entire database. Never underestimate the simplest payloads.



CHAPTER 3

UNION SELECT

Combining Forces

Once an attacker confirms that a parameter is injectable, the `UNION SELECT` technique becomes their primary weapon for data extraction. By appending a second SELECT statement to the original query, attackers can reach across tables and retrieve data they were never authorized to access. This chapter breaks down the mechanics and shows real-world escalation patterns.

The UNION Operator: Merging Query Results

The SQL `UNION` operator is a legitimate, powerful feature of SQL used to combine the result sets of two or more `SELECT` statements into a single output. In normal development, it's used for aggregating data from similar tables. In the hands of an attacker, it becomes a data extraction tool of enormous power.

1

Rule 1: Matching Column Count

Both the original query and the injected `UNION SELECT` must return the **exact same number of columns**. If the original query returns 3 columns, the injected query must also return 3 columns. A mismatch causes a database error, which can actually be useful to the attacker for probing column count.

2

Rule 2: Compatible Data Types

The columns in the injected query must have **compatible data types** with the corresponding columns in the original. In practice, attackers often use `NULL` as a universal placeholder – it is compatible with any data type and avoids type mismatch errors during the probing phase.

3

Rule 3: Data Appears in Output

For the UNION attack to be useful, the application must actually **display the query results** on the page or in the API response. If the app displays data from the injected query, the attacker can see extracted values. This is known as an "in-band" SQL injection – the data travels back through the same channel used to exploit the vulnerability.

Determining Column Count: The First Step

Before an attacker can execute a successful `UNION SELECT` injection, they must determine exactly how many columns the original query returns. There are two primary techniques for this, and both are methodical, iterative, and surprisingly reliable against unprotected applications.

Technique 1: ORDER BY Probing

The attacker increments the `ORDER BY` clause until the application throws an error. The last working number reveals the column count.

```
GET /products?id=1 ORDER BY 1--
GET /products?id=1 ORDER BY 2--
GET /products?id=1 ORDER BY 3--
GET /products?id=1 ORDER BY 4--
-- Error on 4 → 3 columns exist
```

This works because `ORDER BY` can reference columns by position number. Referencing a position beyond the column count triggers an error, giving the attacker their boundary.

Technique 2: NULL Injection Probing

Alternatively, the attacker uses `UNION SELECT NULL` with incrementing NULLs until the query succeeds without an error:

```
UNION SELECT NULL--
UNION SELECT NULL, NULL--
UNION SELECT NULL, NULL, NULL--
-- Success on 3 NULLs → 3 columns
```

Once column count is confirmed, the attacker replaces individual `NULL` values with strings or expressions to identify which columns are rendered visibly in the application's output. For example:

```
UNION SELECT 'a', NULL, NULL--
UNION SELECT NULL, 'a', NULL--
```

Whichever substitution causes `'a'` to appear on the page identifies a **visible, string-compatible column** – the ideal vehicle for extracting real data.

Extracting Specific Data with UNION

With the column count determined and visible columns identified, the attacker is now ready to extract real, sensitive data. This is where a UNION SELECT injection transitions from a proof-of-concept to a full-scale data breach. The example below targets a `users` table containing credentials.

1

Confirm Injection Point

The attacker has already verified that the `id` parameter in `/products?id=1` is injectable and returns 2 visible columns.

2

Craft the Payload

```
1' UNION SELECT
  username, password
FROM users--
```

The attacker targets the `users` table, pulling credentials into the two visible columns of the product query's output.

3

Data Exfiltrated

The application response now includes rows of usernames and hashed (or plaintext) passwords alongside legitimate product records – all rendered in the normal page output.

Escalation note: Once the attacker has credentials, they can attempt to crack password hashes offline, authenticate as privileged users (e.g., admins), pivot to other systems, or leverage database features like `LOAD_FILE()` and `INTO OUTFILE` for file system access – depending on the database configuration and permission level of the database user.

Two Queries, One Goal

The `UNION` operator was designed to merge legitimate datasets. In a SQL injection context, it merges the attacker's desired data with the application's legitimate output – making stolen information appear right alongside normal query results, invisible to a cursory inspection of the interface.



CHAPTER 4

information_schema

The Database's Blueprint

If UNION SELECT is the attacker's extraction tool, `information_schema` is their map. This built-in metadata catalog exists in all major relational database systems and describes the structure of every database on the server – without the attacker needing any prior knowledge of table or column names. It transforms SQL injection from a targeted attack into a comprehensive reconnaissance operation.

What is information_schema?

`information_schema` is a special read-only schema (or virtual database) that is automatically present in all ANSI-compliant relational database management systems, including MySQL, PostgreSQL, MariaDB, and SQL Server. It serves as a **self-describing catalog** – a database about the database itself.

What It Contains

- **SCHEMATA:** Lists all databases available on the server
- **TABLES:** Lists all tables within each database, including table type and row count
- **COLUMNS:** Lists every column in every table, including name, data type, nullability, and default values
- **USER_PRIVILEGES:** Lists database users and their access privileges
- **VIEWS, ROUTINES, TRIGGERS:** Metadata about stored procedures, functions, and triggers

Why It's Crucial for Attackers

In a normal SQL injection scenario, the attacker doesn't know the names of the tables or columns they want to target. Without this knowledge, extracting specific data is like searching blindly in the dark.

`information_schema` eliminates that blindness entirely. By querying its views, an attacker can **enumerate the entire database structure** systematically – discovering every database name, every table, and every column – without any insider knowledge. This makes it the first destination for any attacker who has confirmed a working UNION injection.

The typical attack flow is: **enumerate databases → enumerate tables → enumerate columns → extract target data.** Each step uses `information_schema` to guide the next.

Querying for Database Names

The first step in a systematic SQL injection reconnaissance attack is discovering which databases exist on the server. The `information_schema.schemata` view makes this trivially easy. Each row in this view represents one database (schema) accessible to the current database user.

The Payload

```
' UNION SELECT schema_name
FROM
information_schema.schemata--
```

This payload is injected into a vulnerable parameter. The `--` comment sequence terminates the rest of the original query, preventing syntax errors from trailing SQL code.

What the Attacker Sees

The application's response now includes all database names on the server. A typical MySQL server might return:

```
information_schema
mysql
performance_schema
target_db
customer_db
internal_hr
```

Each returned name is a candidate for further enumeration. The attacker now knows exactly which databases to target next.

Column Count Adjustment

If the original query returns more than one column, the attacker must pad the UNION with NULLs to match. For a 3-column query:

```
' UNION SELECT schema_name,
NULL, NULL
FROM
information_schema.schemata--
```

Only one column needs to carry actual data – the rest can be filled with NULL or literal strings to satisfy the column-count requirement.

Querying for Table Names within a Database

Once the attacker has identified a target database name, the next step is enumerating its tables. The `information_schema.tables` view provides a complete listing of every table in every schema, and can be filtered by database name using a `WHERE` clause.

Payload: List All Tables

```
' UNION SELECT table_name
FROM information_schema.tables
WHERE table_schema
  = 'target_db_name'--
```

Replace `target_db_name` with the actual database name discovered in the previous step. The query returns every table name within that database.

Payload: List All Columns

```
' UNION SELECT column_name
FROM information_schema.columns
WHERE table_name = 'users'
AND table_schema
  = 'target_db_name'--
```

Once a high-value table (like `users`, `accounts`, or `orders`) is identified, this query reveals every column it contains.

Example Output and Escalation

Suppose the attacker queries tables in `target_db` and receives:

```
products
orders
users
admin_settings
payment_info
```

The attacker immediately targets `users` and `payment_info`. They then query columns for each:

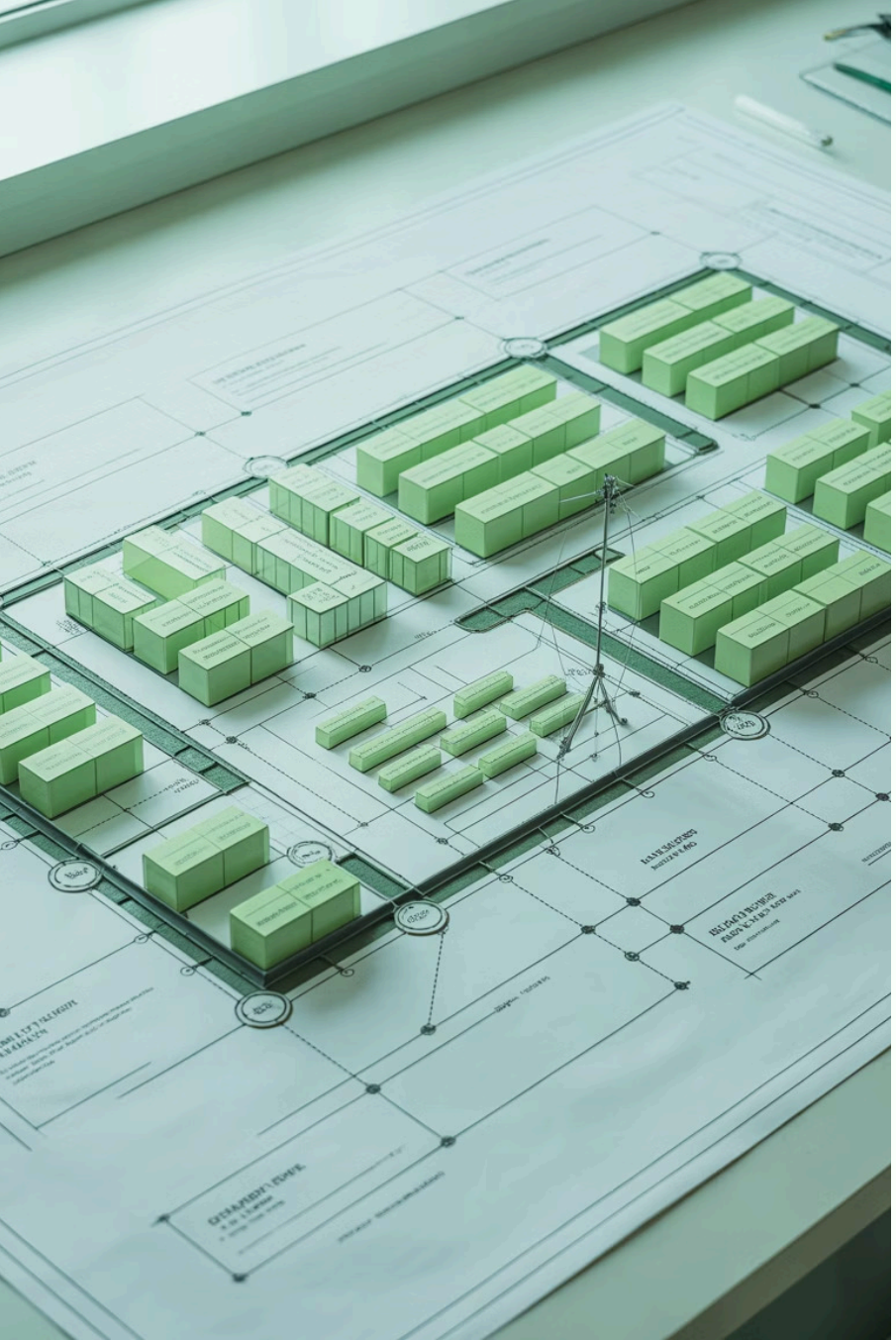
```
-- Columns in 'users':
id, username, email,
password_hash, is_admin

-- Columns in 'payment_info':
id, user_id, card_number,
expiry_date, cvv_hash
```

With database name, table name, and column names all in hand, the attacker now executes a final targeted extraction:

```
' UNION SELECT username,
password_hash
FROM users--
```

This completes the full reconnaissance-to-extraction chain – a structured, methodical attack enabled entirely by `information_schema`. The entire process can take minutes on an unprotected application.



The Attacker's Map

`information_schema` gives every attacker with a working injection point a complete architectural blueprint of your database server – every database, every table, every column. What was once invisible becomes fully legible. Defense requires understanding exactly how thoroughly this metadata exposes your data model.

The Takeaway: Defend Your Digital Doors

SQL Injection is not an obscure or exotic vulnerability – it is one of the oldest, most well-documented, and most exploited attack classes in web security history. Yet it continues to cause breaches at organizations of every size. The good news: it is also almost entirely preventable with the right coding practices.

Parameterized Queries

The single most effective defense is using **prepared statements with parameterized queries**. Instead of concatenating input into SQL strings, you pass parameters separately. The database then treats them as data – never as executable code. This completely eliminates the injection vector, regardless of what the input contains.

```
-- Safe (parameterized):
SELECT * FROM users
WHERE username = ?
AND password = ?
```

Input Validation & Sanitization

Always **validate and sanitize user input** on the server side. Use allowlists (not blocklists) to define what constitutes valid input. Reject or escape special characters like single quotes, double dashes, and semicolons. Never rely on client-side validation alone – it can be bypassed trivially by any attacker using a proxy tool like Burp Suite.

Web Application Firewalls

A **Web Application Firewall (WAF)** can detect and block common SQL injection patterns in HTTP requests before they reach your application. While a WAF is not a substitute for secure coding, it provides an essential additional layer of defense – particularly useful for legacy systems that cannot be easily refactored to use parameterized queries.

Principle of Least Privilege

Configure your database user accounts to have the **minimum permissions necessary**. A web application's database user should not have `DROP TABLE`, `CREATE USER`, or `FILE` privileges. If an attacker does achieve injection, restricted permissions limit the blast radius – preventing them from deleting tables, reading system files, or creating new admin accounts.

Key Takeaway: SQL Injection is preventable. Use parameterized queries as your foundation, validate all input, apply least privilege database permissions, and layer in WAF protections. Security is not a single lock – it is a series of doors, each one making an attacker's job harder.